PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science

Academic Year: 2026-2027
Semester: V
Class / Branch: TE B
Subject: Agile Software Development & DevOps
Name of Instructor: Prof. Sameer Nanivadekar

Name of Student: Gargee Warke
Student ID: 25207001
Date of Performance: 19/10/2026
Date of submit: 20/10/2026

Assignment No. 2

Q1 : Implement a DevOps CI/CD pipeline for an application using GitHub Actions or Jenkins. Configure automated build and testing, containerize the application using Docker, and deploy it using Docker Compose. Demonstrate the execution of the pipeline with suitable screenshots. [LO4, L3]

Implementation:

A College ERP demonstration web application (Python, with automated tests written for pytest) was containerised with Docker, deployed with Docker Compose, and wired to a GitHub Actions workflow so that every push to the main branch is built, tested and deployed automatically. The source is maintained in the GitHub repository for this assignment (link at the end of the document) under Assignment-2/Q1-CI-CD-Docker.

Step 1: Environment setup

Windows Subsystem for Linux (Ubuntu) was installed and Docker was verified before starting. The installed versions were Docker 29.8.2 and Docker Compose v5.6.0, and the docker run hello-world test confirmed that the Docker daemon could pull and run containers.

Step 2: Containerisation with Docker

The application image is defined in a Dockerfile based on python:3.12-slim. The build sets the working directory to /app, copies requirements.txt and installs the dependencies without a pip cache, and then copies the application file (app.py) and the tests folder into the image. A .dockerignore file keeps unnecessary files out of the build context.

Step 3: Automated testing and deployment with Docker Compose

A docker-compose.yml file defines the web service. The tests are executed inside the container using Compose, and the same service then runs the application on port 5000:

```
docker compose -f Assignment-2/Q1-CI-CD-Docker/docker-compose.yml run --build
--rm web python -m pytest -q
```

Step 4: CI/CD with GitHub Actions

The workflow file assignment2-q1-ci-cd.yml is triggered on every push to main. It contains a single job, Build, Test and Deploy with Docker Compose, which builds the Docker image, runs the automated tests and deploys the application with Docker Compose. The run named Modernize Assignment 2 Q1 CI-CD workflow (#3) completed with the status Success.

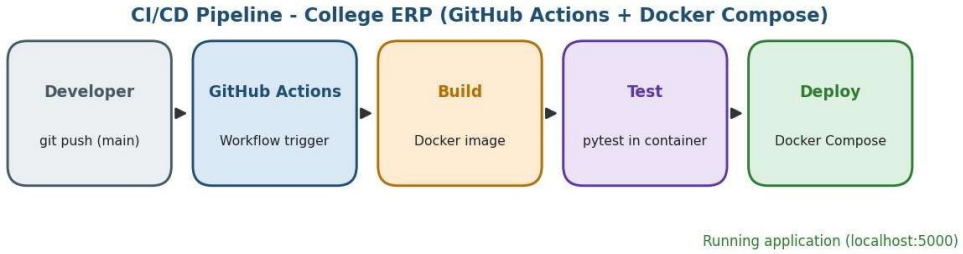


Figure 1: CI/CD pipeline flow for the College ERP application

Output:-

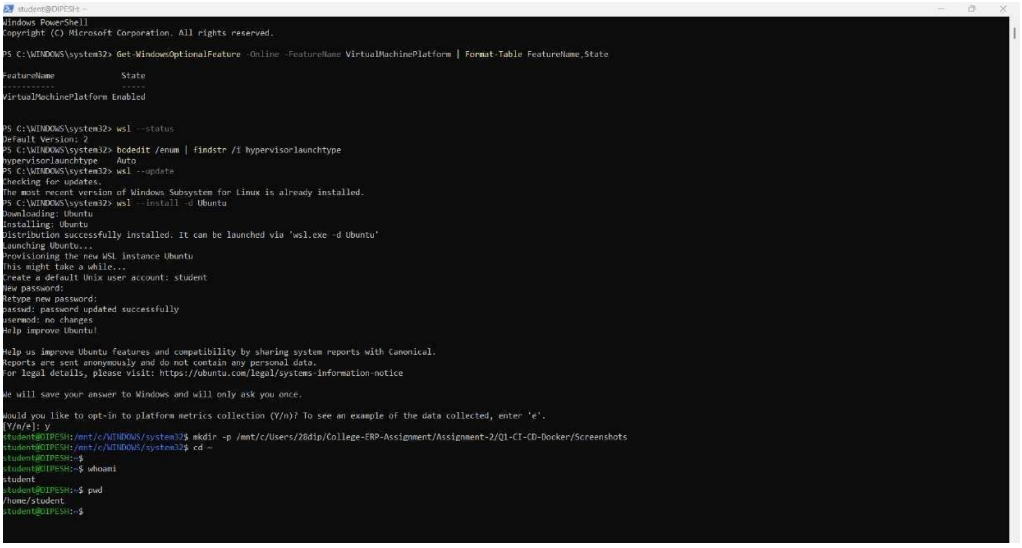


Figure 2: Windows Subsystem for Linux (Ubuntu) installed for the Docker environment

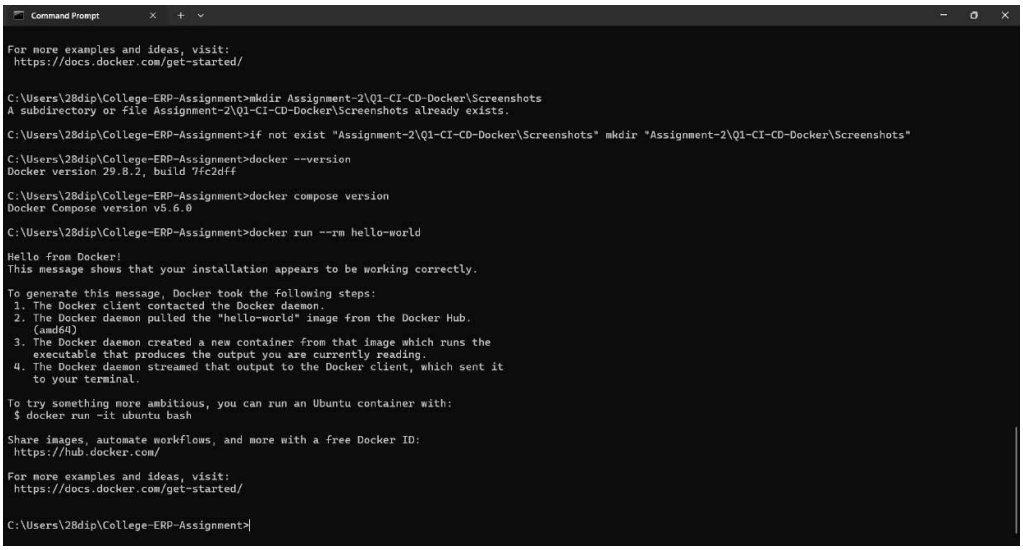


Figure 3: Docker 29.8.2 and Docker Compose v5.6.0 verified with the hello-world container

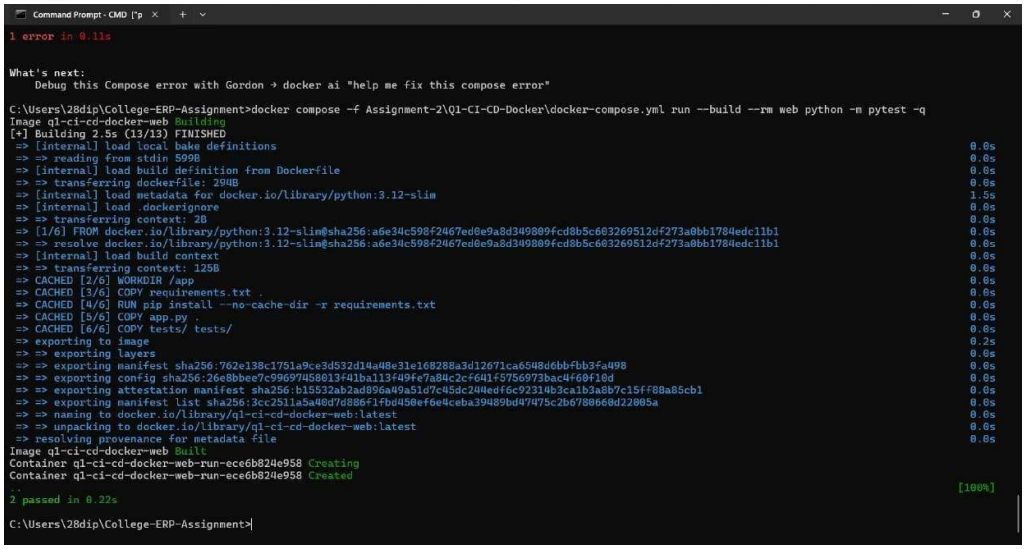


Figure 4: Docker image build and automated tests through Docker Compose (2 passed in 0.22 s)

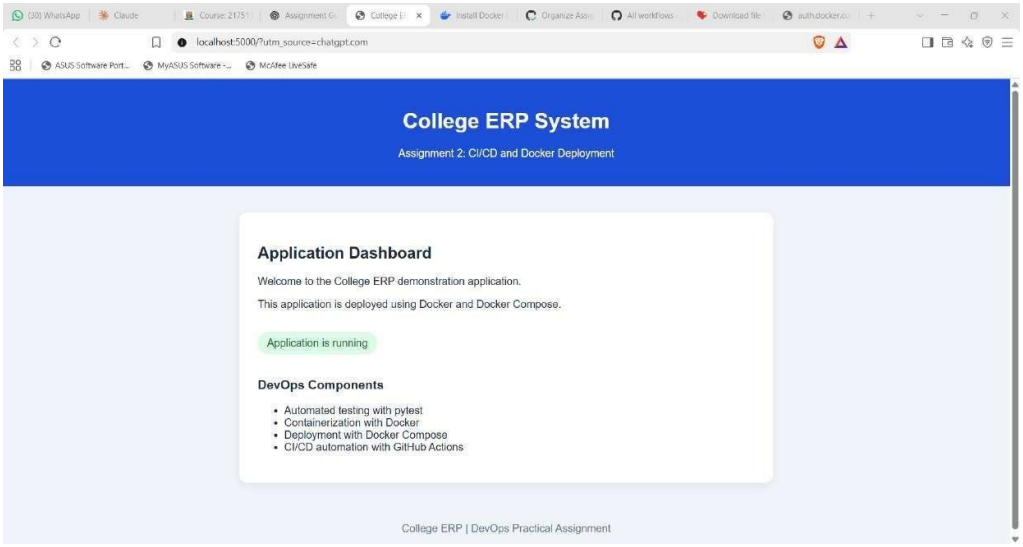


Figure 5: College ERP application deployed by Docker Compose and running on localhost:5000

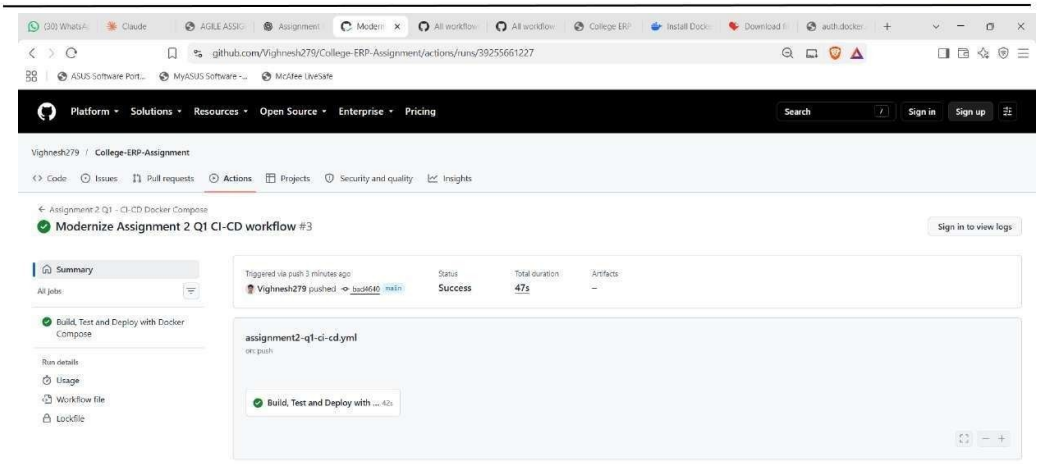


Figure 6: GitHub Actions run #3 completed successfully (job: Build, Test and Deploy with Docker Compose, 47 s)

Q2 : Implement the Machine Learning lifecycle using MLflow by tracking experiments, logging parameters and metrics, managing model versions, and deploying the trained model. Demonstrate basic application monitoring using Prometheus and Grafana. [LO5, L3]

Implementation:

An Iris classification service was built as a Docker Compose stack of five containers, which together cover experiment tracking, model versioning, model serving and monitoring. The work is stored in Assignment-2/Q2-MLflow-Monitoring in the repository.

Service	Image / Port	Role in the ML lifecycle
mlflow	MLflow 2.22.2, port 5001	Tracking server and Model Registry (data kept in the mlflow_data volume)
trainer	Built from the project Dockerfile	Runs train_model.py with MLFLOW_TRACKING_URI=http://mlflow:5001; starts only after MLflow is healthy
api	Built from the project Dockerfile, port 5002	Prediction service exposing the application and a Prometheus metrics endpoint
prometheus	prom/prometheus v3.5.0, port 9090	Scrapes the metrics of the prediction service
grafana	grafana/grafana 12.1.1, port 3000	Dashboards built on the Prometheus data source

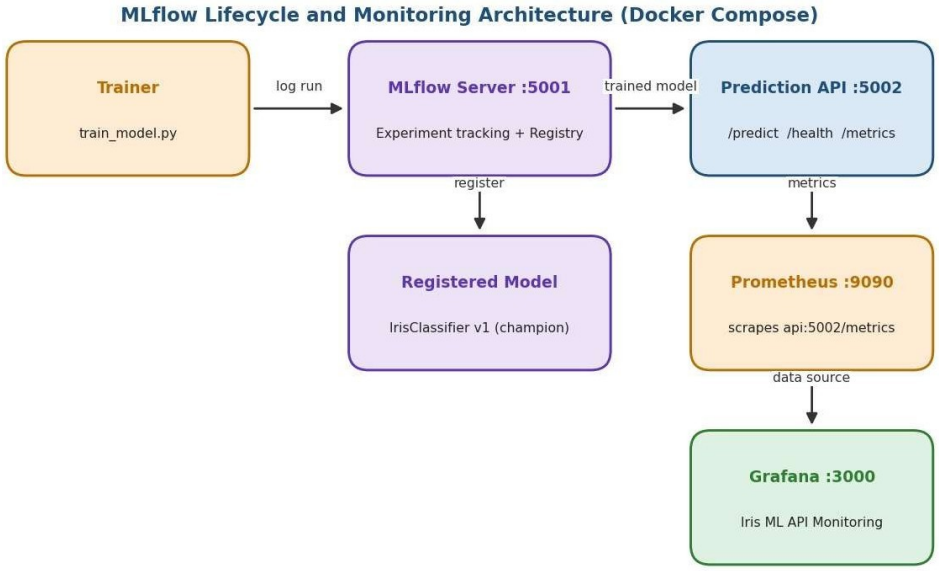


Figure 7: Architecture of the MLflow lifecycle and monitoring stack

Experiment tracking and model versioning

The trainer container connected to the MLflow server and created a run in the experiment named Iris Classification. The trained model was registered in the MLflow Model Registry as IrisClassifier, and Version 1 carries the alias champion, which identifies the version selected for serving.

Model deployment

The prediction API container reports a healthy status in docker compose ps and listens on port 5002. Its automated test-suite (5 tests) covers the home page, the health endpoint, rejection of invalid prediction input, a successful prediction and the Prometheus metrics endpoint; all 5 passed.

Monitoring with Prometheus and Grafana

The API publishes three custom metrics at /metrics: iris_prediction_requests_total, iris_prediction_errors_total and iris_prediction_duration_seconds. Prometheus scrapes the api:5002 target (job iris-api), which is reported as UP, and Grafana displays the dashboard Iris ML API Monitoring with the panels Prediction Requests per Minute, Prediction Errors per Minute and Average Prediction Latency.

Output:-

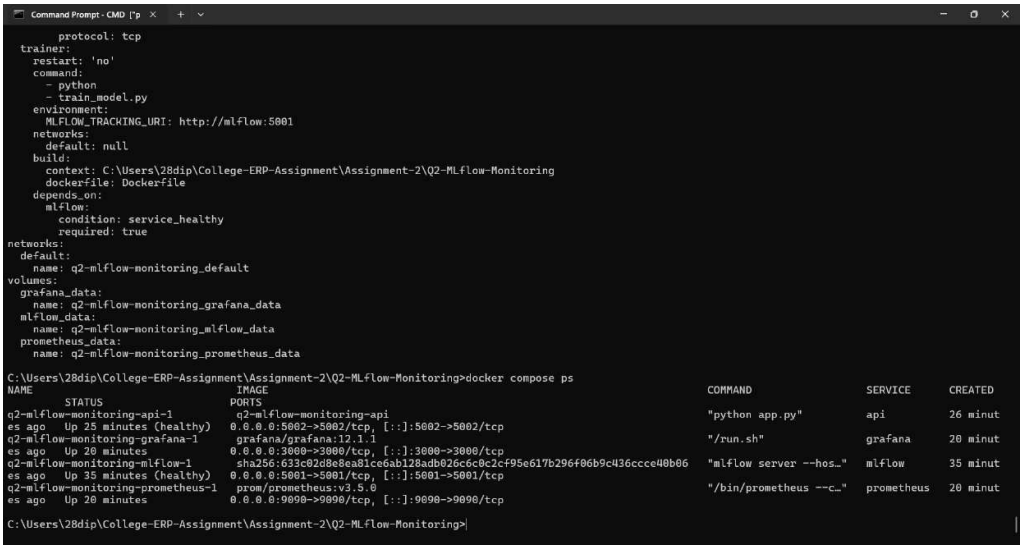


Figure 8: docker compose ps: mlflow, api, prometheus and grafana containers running (api and mlflow healthy)

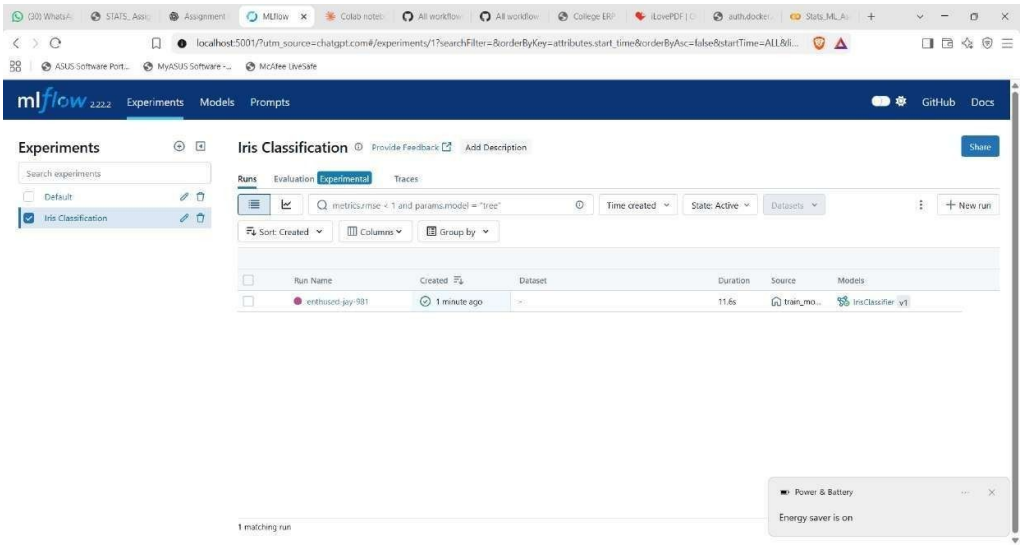


Figure 9: MLflow experiment Iris Classification with the tracked training run and the registered model IrisClassifier v1

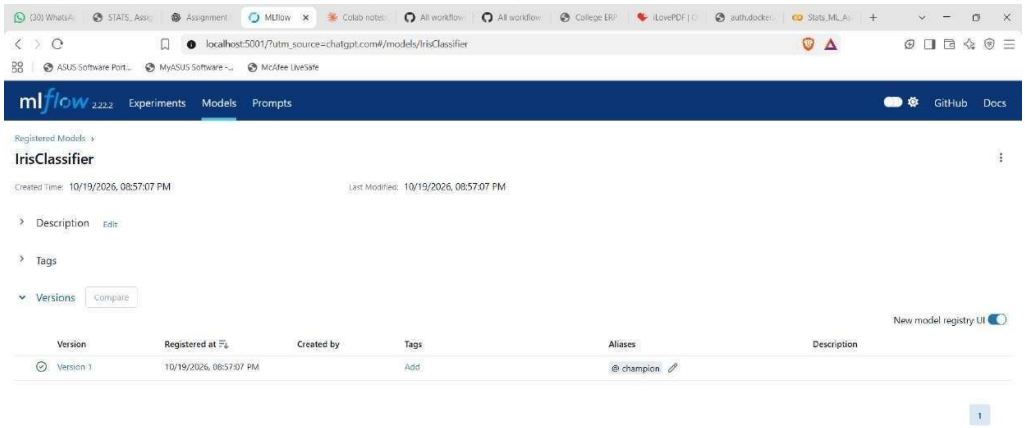


Figure 10: MLflow Model Registry: IrisClassifier Version 1 with the champion alias

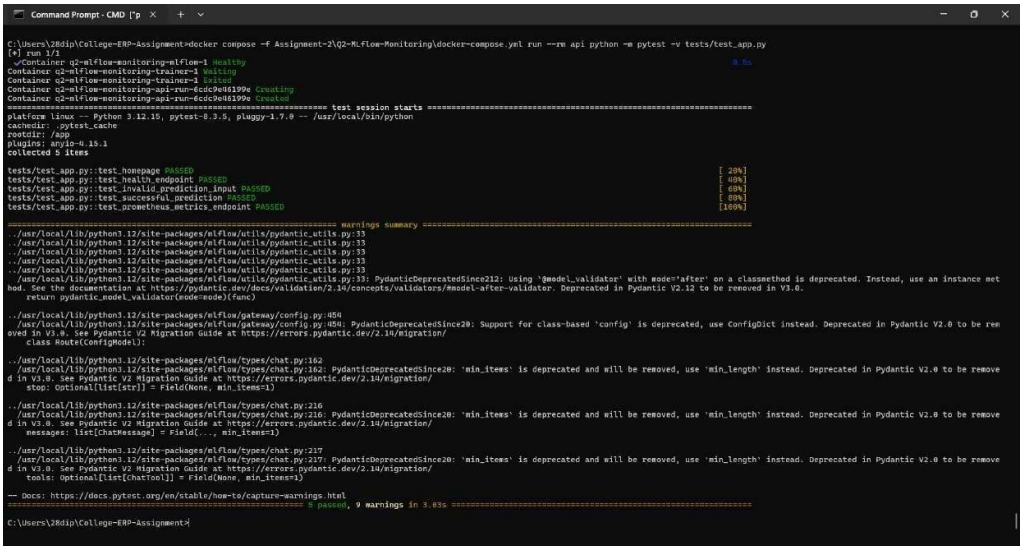


Figure 11: Automated tests of the prediction API: 5 passed

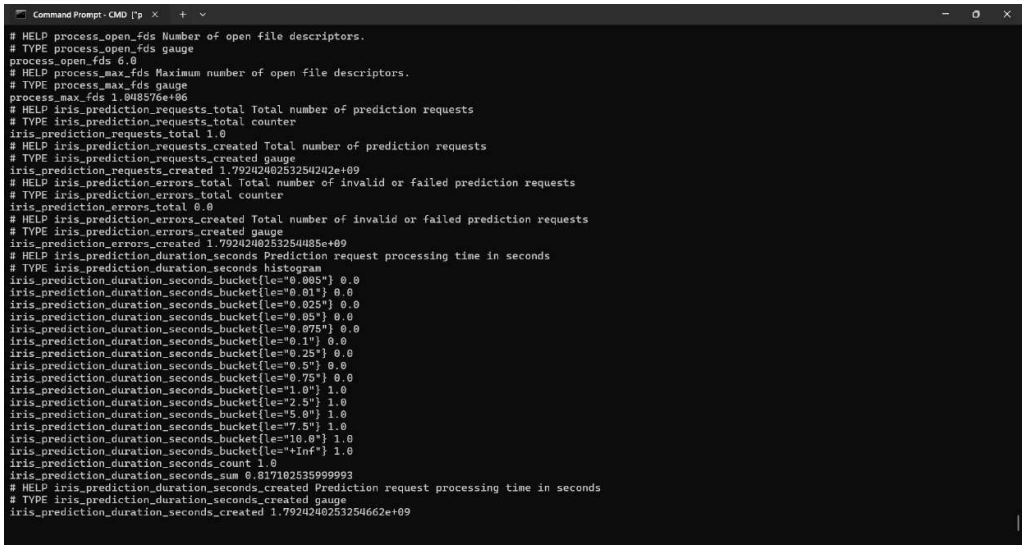


Figure 12: Prometheus metrics exposed by the API (requests, errors and prediction latency histogram)

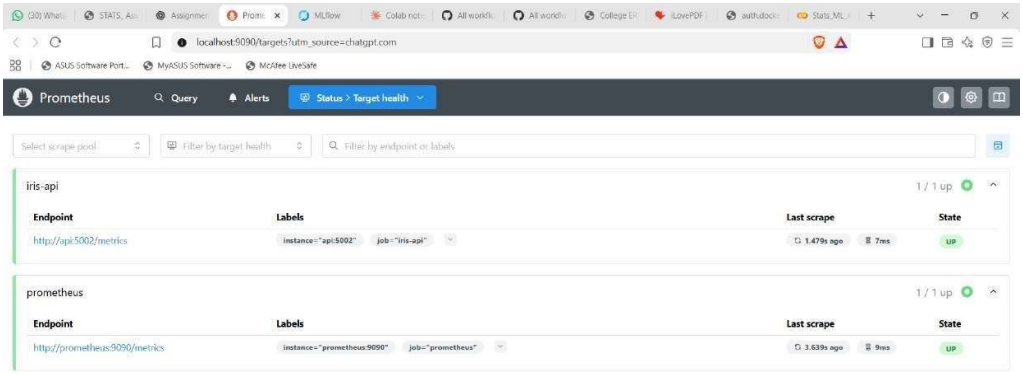


Figure 13: Prometheus target health: iris-api and prometheus targets are UP

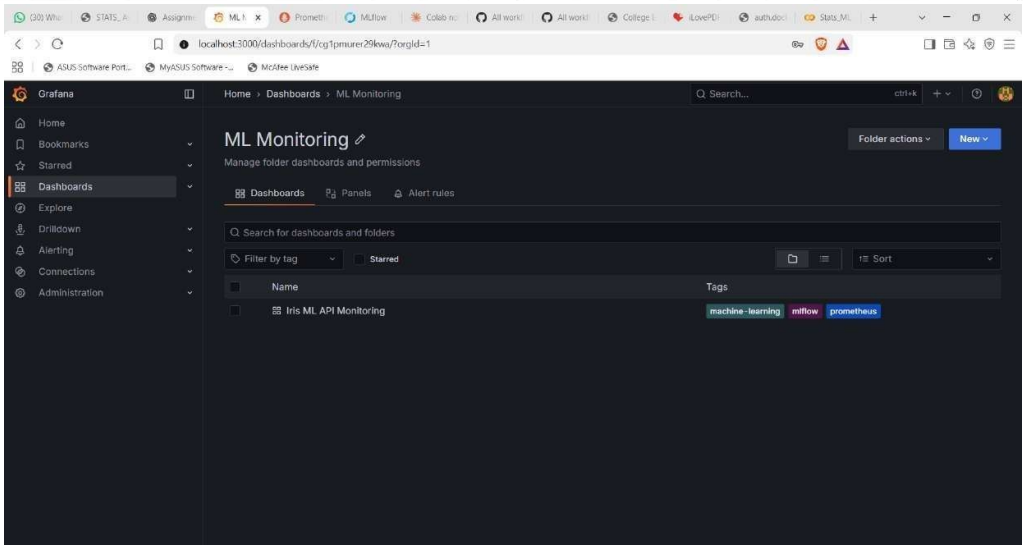


Figure 14: Grafana folder ML Monitoring containing the Iris ML API Monitoring dashboard

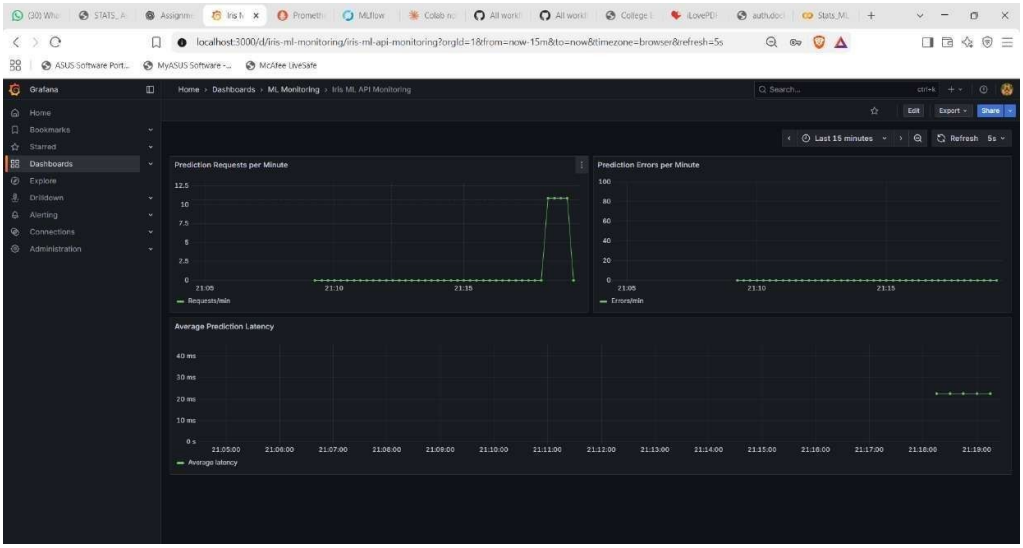


Figure 15: Grafana dashboard: prediction requests per minute, errors per minute and average latency

Q3 : Design, integrate, and demonstrate an end-to-end DevOps pipeline for a selected application by integrating Agile planning, Git-based version control, automated CI/CD, application deployment, and monitoring. Present the complete workflow using an architecture diagram and execution evidence. [LO6, L6]

Implementation:

The Iris ML prediction service from Q2 was selected as the application. The end-to-end pipeline connects the five DevOps stages shown in the architecture diagram below. The artefacts are kept in Assignment-2/Q3-End-to-End-DevOps and in the workflow file q3-end-to-end-devops.yml.

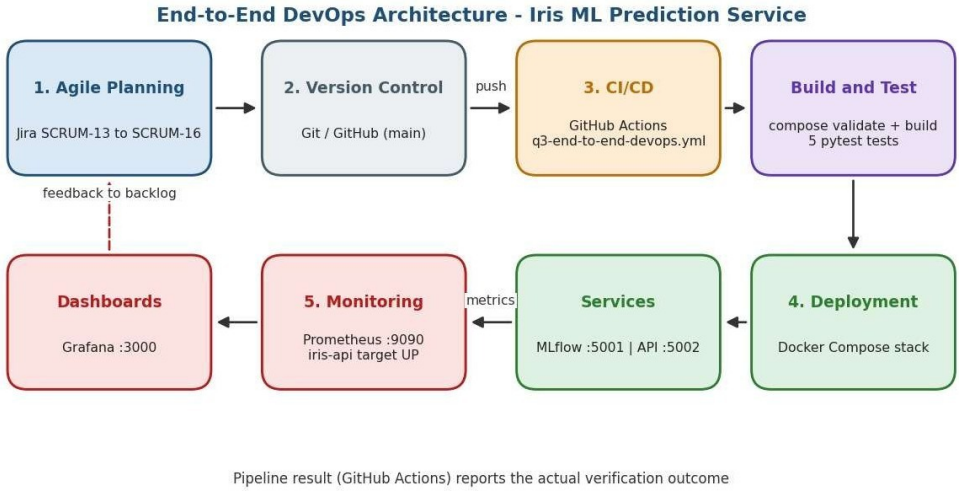


Figure 16: End-to-end DevOps architecture for the Iris ML prediction service

Stage	Tool	Implementation
1. Agile planning	Jira (Scrum project College ERP System)	Story SCRUM-13 Automate CI/CD for the Iris ML Prediction Service with 7 acceptance criteria and sub-tasks SCRUM-14 (configure the Q3 GitHub Actions workflow), SCRUM-15 (verify deployment and application monitoring) and SCRUM-16 (document the end-to-end DevOps architecture)
2. Version control	Git and GitHub	Changes are committed and pushed to the main branch of the College-ERP-Assignment repository
3. CI/CD	GitHub Actions	Workflow q3-end-to-end-devops.yml with one job, Build, Test, Deploy and Monitor, triggered on push
4. Deployment	Docker Compose	Compose configuration is validated and the images are built; MLflow, the trainer, the Flask prediction API, Prometheus and Grafana are started
5. Monitoring	Prometheus and Grafana	The API health check and a sample prediction are verified and the iris-api target is checked for the UP state

Acceptance criteria of story SCRUM-13

1. GitHub Actions automatically triggers when relevant project files are changed.
2. Docker Compose configuration is validated and the application images build successfully.
3. All five automated application tests pass.
4. The MLflow, Flask API, Prometheus and Grafana services start successfully.
5. The prediction API health check and a sample prediction request succeed.
6. Prometheus reports the iris-api target as UP.
7. The pipeline reports the actual verification result in GitHub Actions.

Output:-

Agile planning in Jira

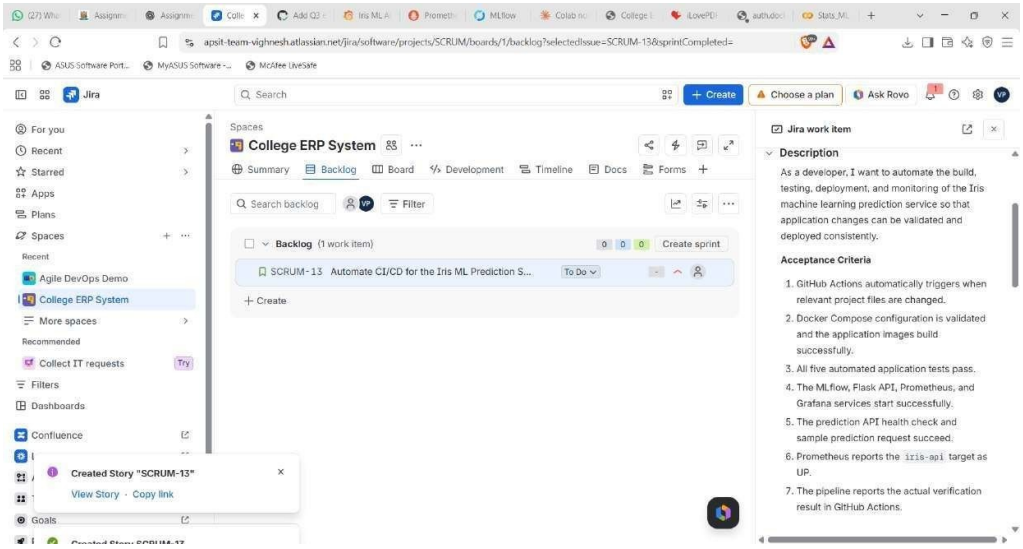


Figure 17: Jira story SCRUM-13 with its description and acceptance criteria

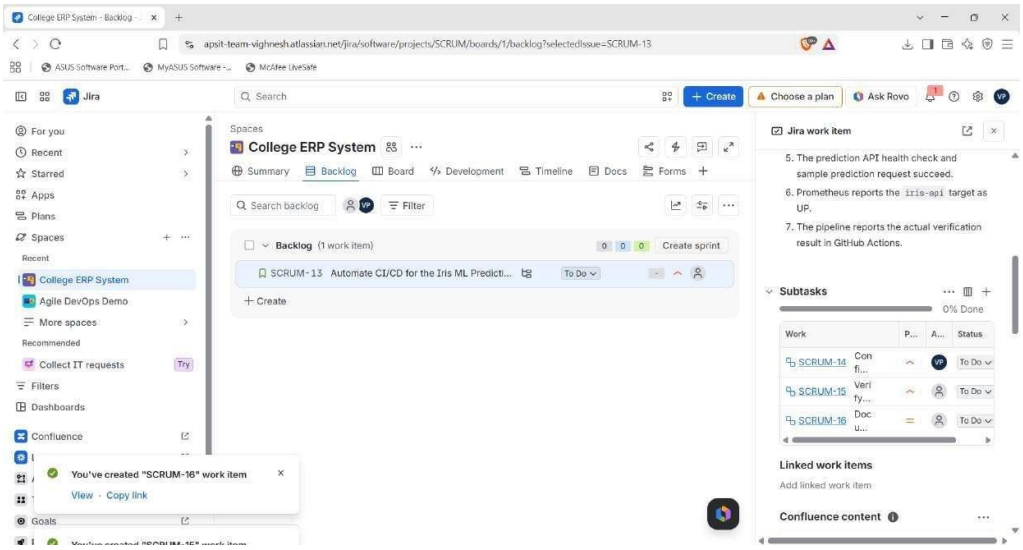


Figure 18: Sub-tasks SCRUM-14, SCRUM-15 and SCRUM-16 created under SCRUM-13

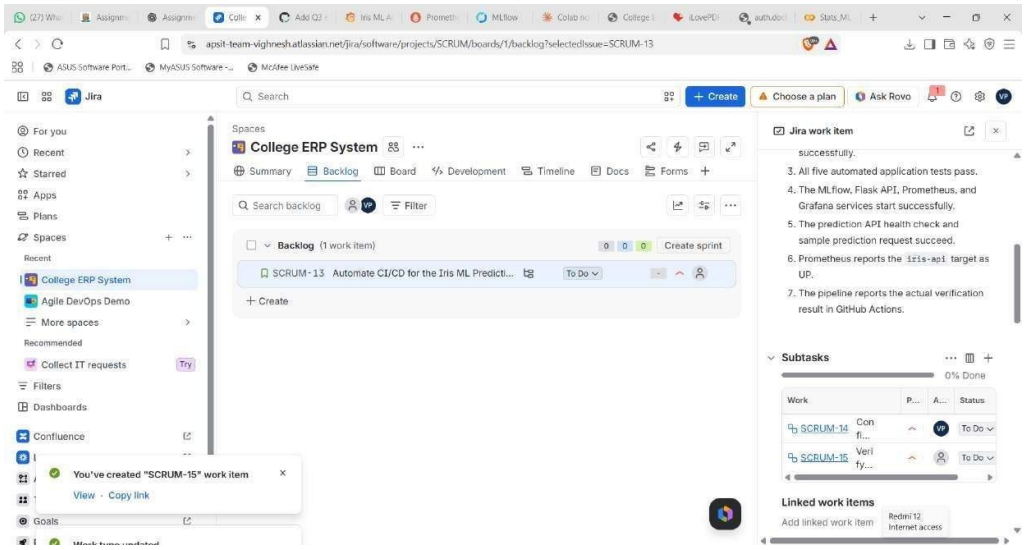


Figure 19: Product backlog showing SCRUM-13 and its sub-tasks (SCRUM-14 assigned)

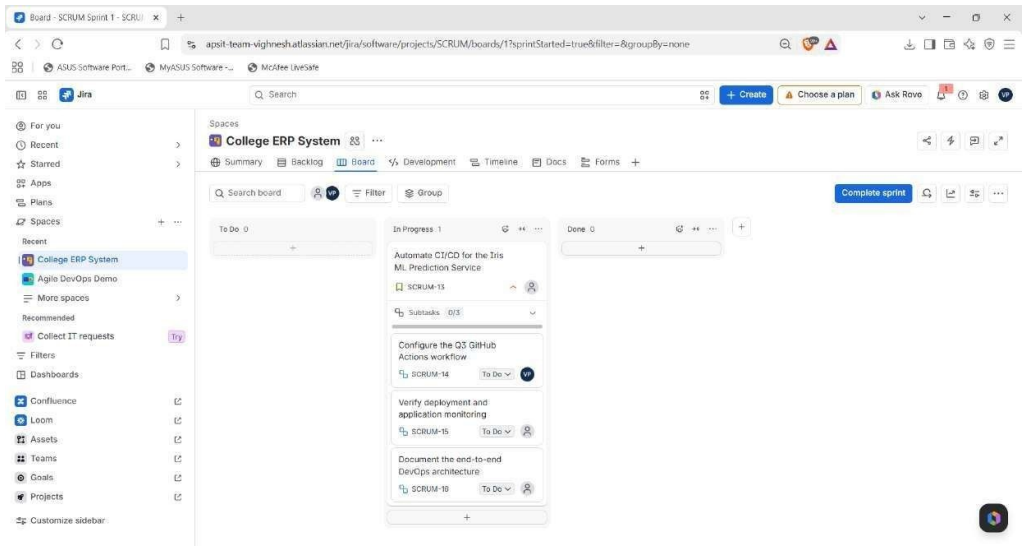


Figure 20: Scrum board at the start: story in progress, all three sub-tasks in To Do (0/3)

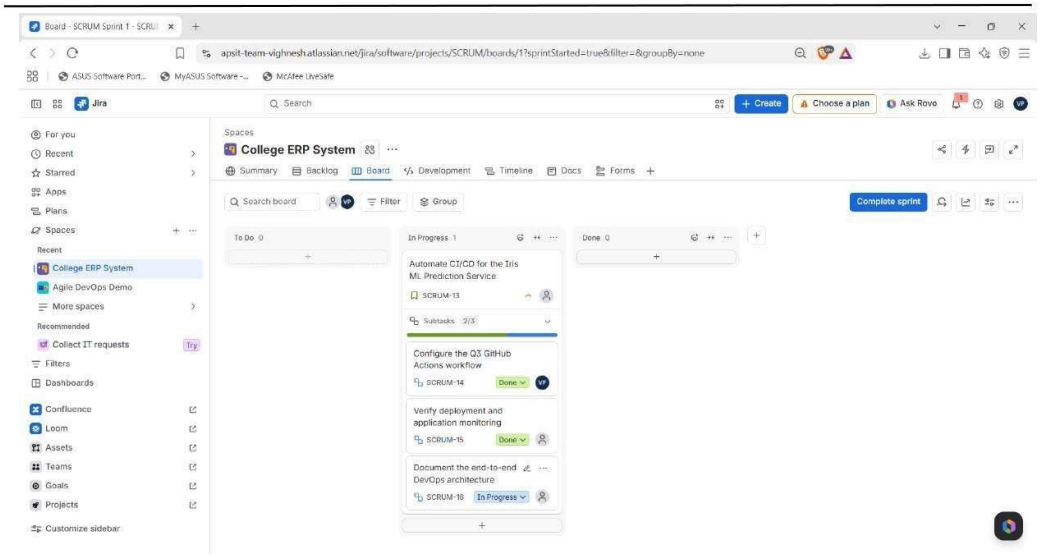


Figure 21: Scrum board after work progressed: SCRUM-14 and SCRUM-15 Done (2/3), SCRUM-16 in progress

Automated CI/CD execution on GitHub Actions

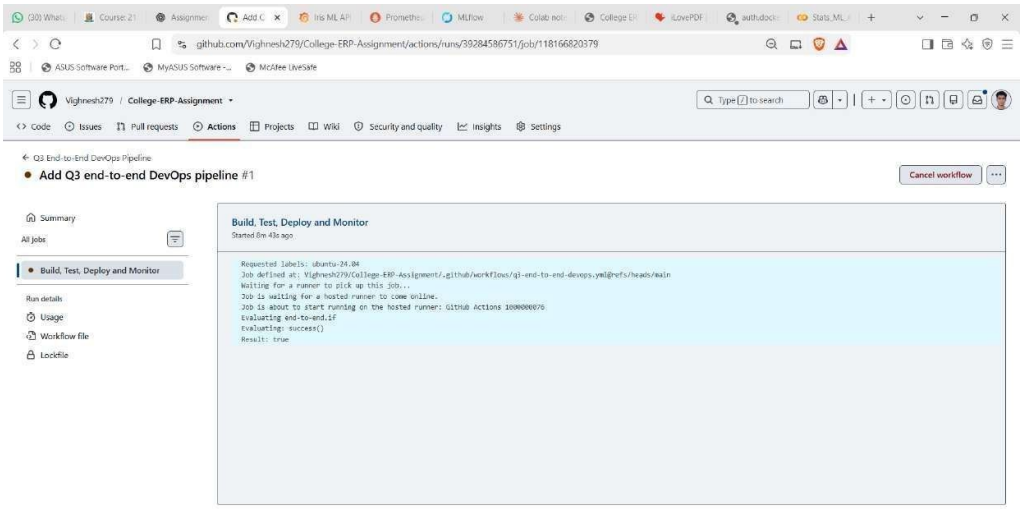


Figure 22: Workflow Add Q3 end-to-end DevOps pipeline #1 started: the job Build, Test, Deploy and Monitor is running

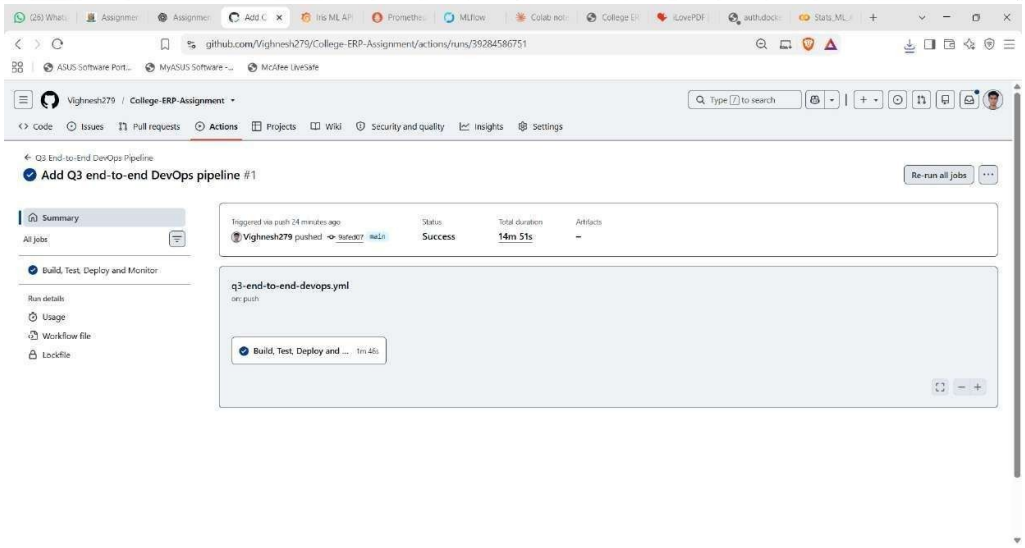


Figure 23: Workflow run #1 completed with the status Success (job duration 1 min 46 s)

Deployment and monitoring of the application

The pipeline deploys the same containerised stack that was demonstrated in Q2. The running services and the monitoring dashboard are shown below.

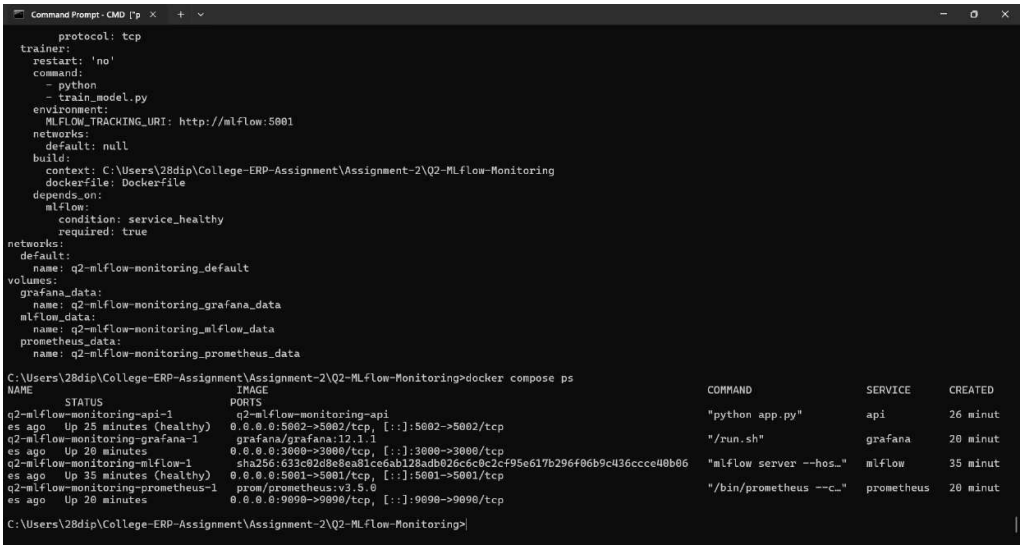


Figure 24: Deployed services: MLflow, prediction API, Prometheus and Grafana running under Docker Compose

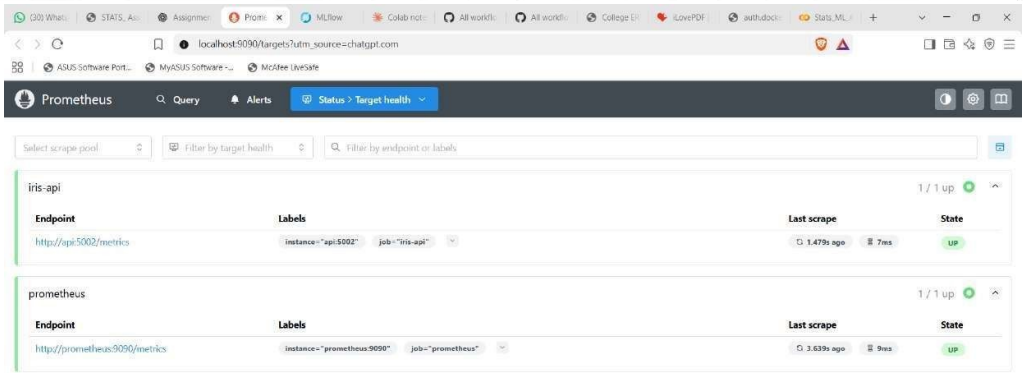


Figure 25: Prometheus confirms that the iris-api target is UP

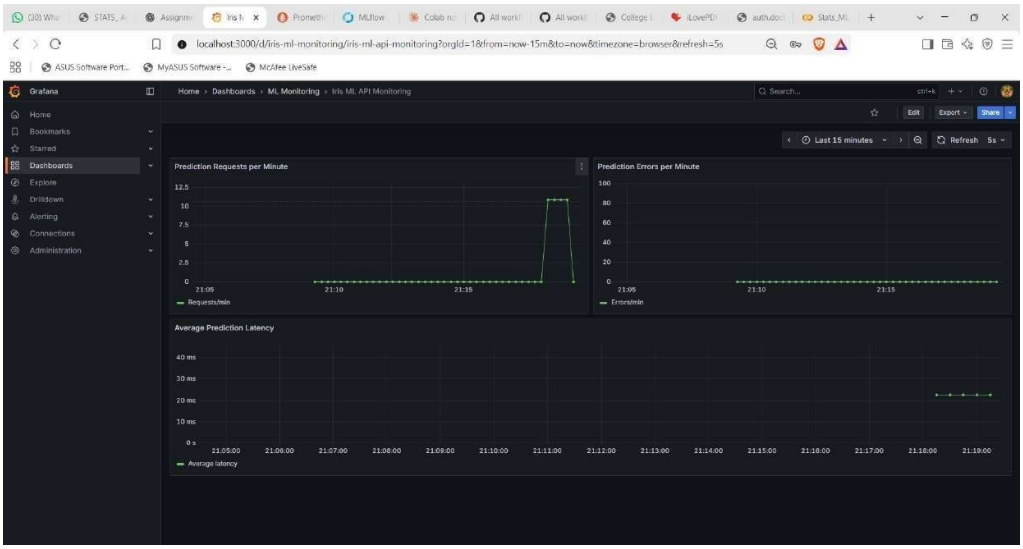


Figure 26: Grafana dashboard monitoring requests, errors and latency of the deployed service

Conclusion:

Assignment 2 demonstrates a complete DevOps workflow: the application is packaged with Docker and deployed with Docker Compose (Q1), the machine learning lifecycle is tracked, versioned and monitored with MLflow, Prometheus and Grafana (Q2), and Agile planning in Jira, Git-based version control, an automated GitHub Actions pipeline, deployment and monitoring are integrated into one end-to-end process (Q3).

GitHub repository: https://github.com/Vighnesh279/College-ERP-Assignment